

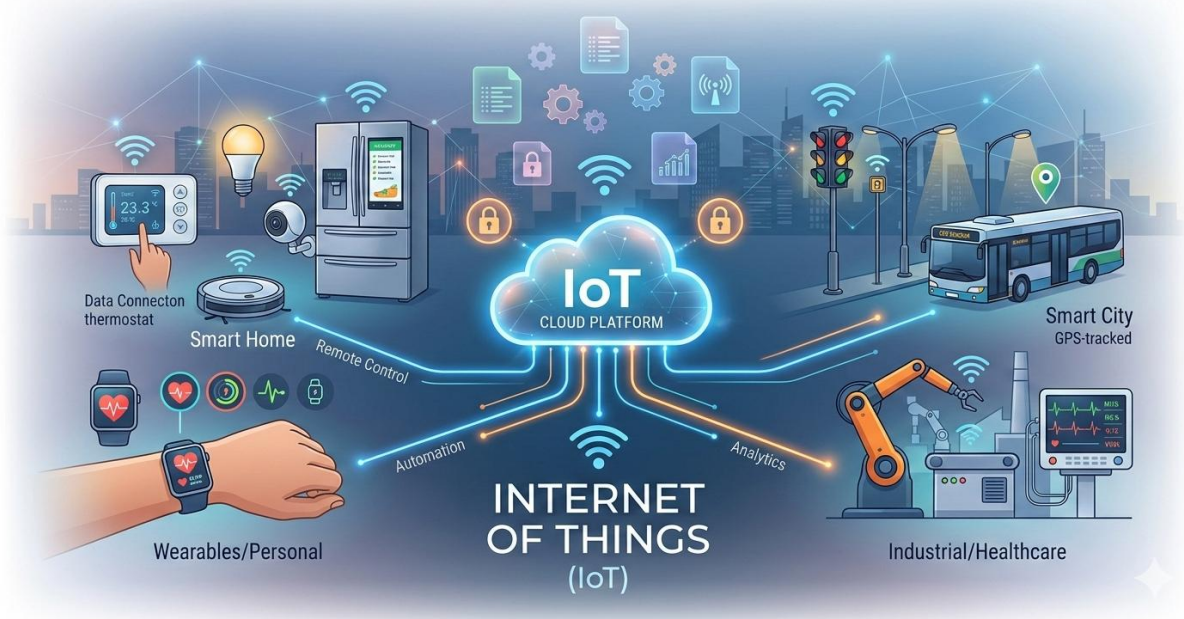
RAPPORT DE GESTION DE PROJET AVANCE

CYCLE D'INGENIEUR

**ÉCOLE SUPERIEURE DE GENIE BIOMEDICAL ET DES TECHNIQUES DE SANTE
SUPTECH SANTE**

GENIE DIGITALE ET INTELLIGENCE ARTIFICIELLE EN SANTE

IOT



Nom de l'étudiant :

ERRIFAIY JASSIM

Encadrant professionnel :

PR BELHADAOU

Année universitaire 2025/2026

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Je remercie particulièrement mon encadrant professionnel, PR BELHADAoui, pour son accompagnement, ses conseils précieux et son soutien tout au long de ce travail. Ses orientations et son expertise ont été d'une grande importance pour mener à bien ce projet.

J'adresse également mes sincères remerciements à tous les enseignants et responsables pédagogiques pour la qualité de la formation et les connaissances acquises durant mon parcours, qui ont largement contribué à la réalisation de ce projet.

Enfin, je tiens à remercier toutes les personnes qui m'ont soutenu de près ou de loin dans l'élaboration de ce travail.

RESUME

Ce projet s'inscrit dans le cadre de la transformation digitale des soins hospitaliers (Santé 4.0). L'objectif est de concevoir un système complet de surveillance des constantes vitales (fréquence cardiaque, température, SpO2 et pression artérielle) basé sur l'Internet des Objets (IoT) et l'Intelligence Artificielle.

L'architecture repose sur un microcontrôleur **ESP32** simulant l'acquisition des données médicales, transmises via le protocole **MQTT** vers un broker cloud. L'interface de supervision a été développée sous **Node-RED**, permettant un monitoring en temps réel et un système d'alertes critiques (visuelles et sonores) pour le personnel soignant. Au-delà de la simple visualisation, le projet intègre une dimension prédictive grâce à un moteur d'analyse développé en **Python**, capable d'identifier des tendances physiologiques anormales avant le franchissement des seuils d'alerte. Les données sont systématiquement historisées au format **CSV**, constituant ainsi une base de données exploitable pour l'amélioration continue des soins et l'entraînement de futurs modèles de Machine Learning.

Table de Matière

CHAPITRE I :	7
INTRODUCTION GENERALE	7
I. 1. INTRODUCTION.....	8
I. 1. 1. Contexte Global.....	9
I. 1. 2. Problématique	9
I. 1. 3. Objectif du projet	9
I. 1. 4. Méthodologie	9
CHAPITRE II :	10
CADRAGE, MANAGEMENT ET CONCEPTION DU PROJET	10
II. 1. Organisation des tâches : Le WBS (Work Breakdown Structure).....	11
II. 2. Planification temporelle : Le diagramme de GANTT	11
II. 3. Analyse fonctionnelle : La Matrice DPF	12
II. 4. Définition du périmètre : Le Diagramme de Contexte	13
II. 5. Architecture du système : Diagramme Physique.....	15
CHAPITRE III :	16
III. 1. Architecture Matérielle (Simulation Wokwi).....	17
III. 1. 1. Le Microcontrôleur : ESP32	17
III. 1. 2. Les Capteurs de Constantes Vitales.....	17
III. 2. Environnement Logiciel	19
III. 2. 1. Wokwi : Simulateur IoT.....	19
III. 2. 2. Node-RED : Plateforme de Supervision	19
III. 2. 3. Broker MQTT : HiveMQ	20
III. 2. 4. Python : Analyse et Intelligence Artificielle	20
CHAPITRE IV :	21
IV. 1. Architecture Globale du Système	22
IV. 1. 1. Couche de Perception (Edge) :	22
IV. 1. 2. Le code :.....	23
IV. 1. 3. Couche de Transport (Cloud) :	26
IV. 1. 4. Couche d'Application (User Interface & AI) :	26
IV. 1. 5. Résultat :	28
IV. 2. Le Protocole MQTT :	28

IV. 2. 1. Le code :	29
IV. 2. 2. Résultat :	30
IV. 2. 3. Organisation des Flux (Topics).....	30
IV. 2. 4. Format de Données : Le JSON	30
IV. 2. 5. Fiabilité et Sécurité (QoS)	31
CHAPITRE V :	32
V. 1. Philosophie de conception sous Node-RED	33
V. 1. 1. Nœud de Réception (Entrée)	33
V. 1. 2. Nœuds de Filtrage (Change Nodes).....	33
V. 1. 3. Nœud de Traitement Intelligent (Function).....	34
V. 1. 4. Nœud de Formatage de Données (CSV).....	34
V. 1. 5. Nœud de Stockage (File out).....	35
V. 1. 6. Nœuds d'Interface (Dashboard)	35
V. 2. Architecture du flux de données (Flow)	36
V. 2. 1. Nœud MQTT In :	36
V. 2. 2. Nœud JSON :	36
V. 2. 3. Nœuds de Fonction (Switch/Change) :.....	36
V. 2. 4. Composants du Dashboard Médical	36
V. 3. Logique d'Alerte et de Sécurité	37
CHAPITRE VI :	38
VI. 1. Stratégie d'Historisation (Data Logging)	39
VI. 2. Module d'Analyse Prédictive en Python.....	39
VI. 2. 1. L'Algorithme de Fenêtre Glissante (Sliding Window).....	39
VI. 2. 2. Détection de Tendance (Trend Analysis).....	39
VI. 3. Vers une Médecine Proactive	39
VI. 4. Conclusion Générale et Perspectives	40
VI. 4. 1. Bilan du Projet	40
VI. 4. 2. Apports Personnels	40
VI. 4. 3. Perspectives d'Évolution	40

Figure 1: Work Breakdown Structure du projet Smart Hospital	11
Figure 2 : Planning de réalisation et jalons du projet.	12
Figure 3 : Matrice DPF liant les besoins utilisateurs aux fonctions de service.	13
Figure 4 : Diagramme de contexte du système de surveillance intelligente.....	14
Figure 5 : Architecture physique et schéma de connectivité IoT.	15
Figure 6 : ESP32.....	17
Figure 7 : DS18B20	18
Figure 8 : DHT22	18
Figure 9 : Simulateurs Physiologiques (Potentiomètres)	19
Figure 10 : Node-RED	19
Figure 11 : Le broker public de HiveMQ	20
Figure 12 : Architecture du système ESP32 et les capteurs et le code utiliser sous dessous	22
Figure 13 : Couche de Transport (Cloud)	26
Figure 14 : Code utiliser pour le modèle de AI	27
Figure 15 : Code utiliser pour le subscriber	29
Figure 16 : Architecture logicielle au sein de Node-RED	33
Figure 17 : Le fichier CSV sauvegarder sur notre path	35
Figure 18 : Interface (Dashboard)	36

CHAPITRE I :
INTRODUCTION GENERALE

I. 1. INTRODUCTION

Dans le domaine de la santé, la surveillance des signes vitaux joue un rôle fondamental dans l'évaluation et le suivi de l'état de santé d'un patient. Les signes vitaux représentent les indicateurs physiologiques essentiels du fonctionnement du corps humain. Parmi les plus importants figurent la fréquence cardiaque, la température corporelle, la pression artérielle et la saturation en oxygène du sang (SpO₂). Ces paramètres permettent aux professionnels de santé d'évaluer rapidement la condition clinique d'un patient et de détecter toute anomalie pouvant indiquer une dégradation de son état de santé.

Traditionnellement, la surveillance des signes vitaux est réalisée par le personnel médical à l'aide d'équipements spécialisés dans les hôpitaux ou les centres de soins. Cependant, ces méthodes nécessitent souvent une présence humaine constante et peuvent entraîner des délais dans la détection de certaines anomalies, surtout lorsque le nombre de patients est élevé. De plus, la collecte manuelle des données peut parfois être sujette à des erreurs ou à un manque de continuité dans le suivi des patients.

Avec les progrès technologiques récents, notamment dans les domaines de l'Internet des objets (IoT), des systèmes embarqués et des technologies de communication sans fil, il est désormais possible de concevoir des systèmes intelligents capables de surveiller automatiquement les paramètres vitaux des patients. Ces systèmes utilisent des capteurs biomédicaux connectés à des microcontrôleurs pour mesurer les données physiologiques, puis les transmettre à distance vers des plateformes de supervision et d'analyse. Cette approche permet une surveillance continue et en temps réel, ce qui améliore considérablement la réactivité du personnel médical face aux situations critiques.

L'intégration des technologies IoT dans le secteur de la santé ouvre également la voie à de nouvelles formes de soins, telles que la télémédecine et la surveillance médicale à distance. Grâce à ces solutions, les médecins peuvent accéder aux données des patients à tout moment et depuis n'importe quel endroit, ce qui facilite le diagnostic, le suivi et la prise de décision médicale. De plus, ces systèmes contribuent à réduire la charge de travail du personnel hospitalier tout en améliorant la qualité et l'efficacité des soins.

Dans ce contexte, le développement d'un système de surveillance des signes vitaux basé sur des technologies IoT représente une solution innovante et prometteuse. Un tel système repose généralement sur l'utilisation de capteurs pour mesurer les paramètres physiologiques, d'un microcontrôleur pour traiter les données et de protocoles de communication tels que MQTT pour transmettre les informations vers une plateforme de visualisation comme Node-RED. Les données peuvent ensuite être affichées sous forme de tableaux de bord permettant au personnel médical de surveiller facilement l'état des patients et de recevoir des alertes en cas de valeurs anormales.

Ainsi, la mise en place d'un système intelligent de surveillance des signes vitaux permet non seulement d'améliorer la sécurité des patients, mais aussi d'optimiser la gestion des ressources dans les établissements de santé. Ce type de solution représente une étape importante vers la transformation numérique du secteur médical et le développement d'hôpitaux plus connectés et plus performants.

I. 1. 1. Contexte Global

Le secteur de la santé traverse aujourd'hui une transformation sans précédent, portée par l'émergence de la **Santé 4.0**. Face au vieillissement de la population et à la pression croissante sur les infrastructures hospitalières, la surveillance traditionnelle "au chevet du patient" montre ses limites. L'intégration des technologies de l'**Internet des Objets (IoT)** et de l'**Intelligence Artificielle (IA)** devient alors une nécessité stratégique pour assurer un suivi continu, précis et délocalisé des patients, garantissant ainsi une meilleure réactivité des soins.

I. 1. 2. Problématique

Malgré les avancées technologiques, de nombreux systèmes de monitoring restent encore purement réactifs : ils n'alertent le personnel soignant que lorsque le seuil critique est déjà franchi. De plus, la fragmentation des données et le manque d'historisation automatique empêchent souvent une analyse approfondie des tendances physiologiques. Comment alors concevoir une architecture capable non seulement de transmettre des constantes vitales en temps réel, mais aussi d'anticiper des dégradations de l'état de santé grâce à l'analyse de données ?

I. 1. 3. Objectif du projet

L'objectif principal de ce projet est de réaliser un **système intelligent de surveillance hospitalière**. Ce travail vise à :

- Concevoir une unité d'acquisition de données simulant des constantes vitales (Fréquence cardiaque, Température, SpO2, Pression).
- Mettre en place une communication sécurisée et fluide via le protocole **MQTT**.
- Développer un **Dashboard interactif** pour une visualisation centralisée des données.
- Implémenter un module de **Machine Learning** (analyse de tendances) pour passer d'une surveillance réactive à une médecine prédictive.

I. 1. 4. Méthodologie

Pour mener à bien ce projet, nous avons adopté une approche modulaire basée sur des outils de pointe en ingénierie digitale :

- **Simulation matérielle** : Utilisation de la plateforme **Wokwi** pour simuler un microcontrôleur **ESP32** et ses capteurs.
- **Infrastructure réseau** : Utilisation du broker **HiveMQ** pour la gestion des messages MQTT.
- **Supervision et Logique** : Développement de l'interface et des alertes sous **Node-RED**.
- **Analyse et IA** : Utilisation du langage **Python** pour le traitement statistique et la prédiction de tendances.

CHAPITRE II :

**CADRAGE, MANAGEMENT ET CONCEPTION DU
PROJET**

La réussite d'un projet technologique dans le secteur de la santé ne repose pas uniquement sur la programmation, mais sur une planification rigoureuse et une analyse fonctionnelle précise. Ce chapitre présente les outils de gestion de projet qui ont structuré notre démarche.

II. 1. Organisation des tâches : Le WBS (Work Breakdown Structure)

Afin de maîtriser la complexité du système, nous avons décomposé le projet en unités de travail gérables à travers un organigramme technique (WBS). Cette décomposition permet d'assurer qu'aucun aspect, de l'acquisition des données à la couche d'intelligence artificielle, n'a été négligé.

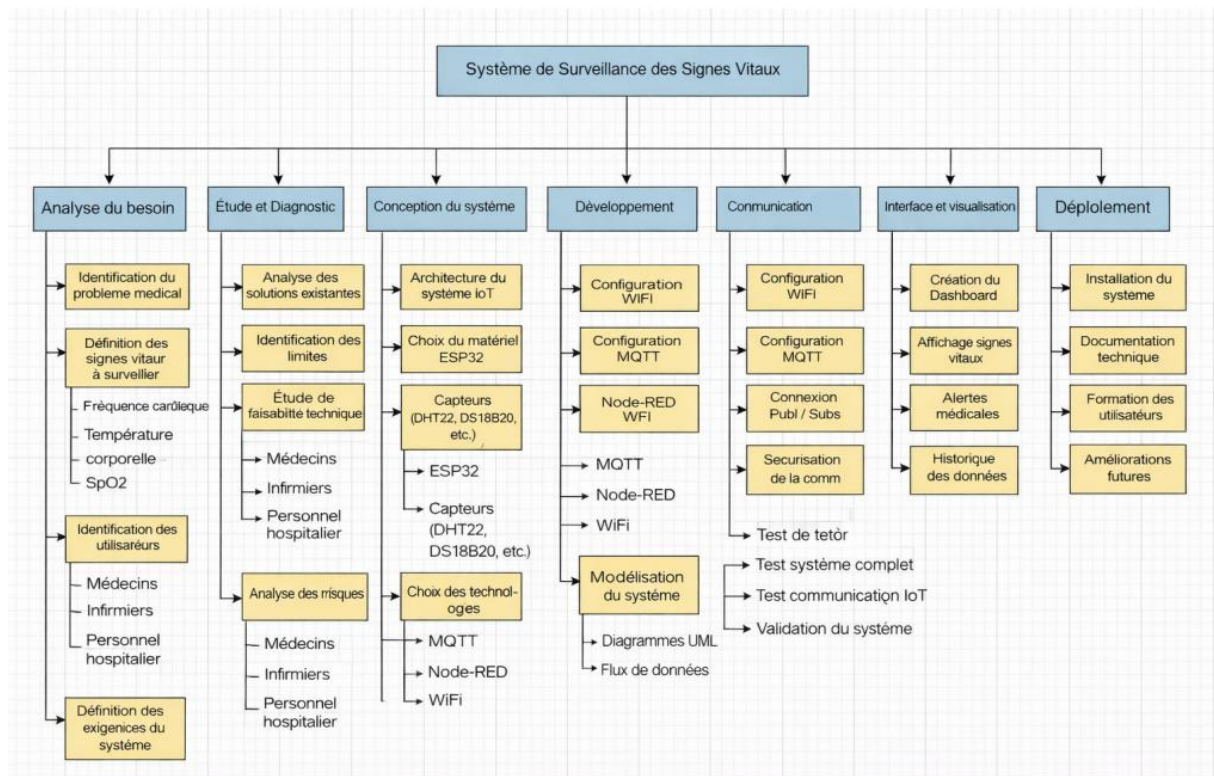


Figure 1: Work Breakdown Structure du projet Smart Hospital

II. 2. Planification temporelle : Le diagramme de GANTT

La gestion du temps a été cruciale pour respecter les échéances académiques de ce quatrième semestre (S4). Le diagramme de Gantt nous a permis de séquencer les étapes de développement, en accordant une importance particulière à la phase d'interconnexion entre le simulateur Wokwi et le broker MQTT.

Tâche	Semaine 1	Semaine 2	Semaine 3	Semaine 4
Analyse				
Conception				
Développement				
Tests				
Documentation				
N°	Phase	Tâche	Durée / h	
1	Analyse	Analyse du besoin	4	
2	Analyse	Identification des utilisateurs	1	
3	Analyse	Définition des exigences	2	
4	Conception	Conception architecture	3	
5	Conception	Définition topics MQTT	1	
6	Conception	Création diagrammes	1	
7	Développement	Programmation ESP32	3	
8	Développement	Configuration MQTT	2	
9	Développement	Développement Node-RED	2	
10	Développement	Implémentation alertes	2	
11	Test	Test communication	1	
12	Test	Test scénarios critiques	1	
13	Documentation	Rédaction rapport	10	
14	Documentation	Préparation soutenance	7	

Figure 2 : Planning de réalisation et jalons du projet.

II. 3. Analyse fonctionnelle : La Matrice DPF

La **Matrice de Décomposition des Processus Fonctionnels (DPF)** constitue le pont entre les besoins cliniques et les solutions techniques. Elle permet de justifier chaque module développé en fonction d'un besoin utilisateur précis (ex: surveillance du rythme cardiaque → fonction de filtrage Node-RED → affichage Gauge).

E2	Détection ouverture fenêtre	Détection automatique de l'ouverture des fenêtres.	F2	Capteur fenêtre	Capteur d'ouverture installé sur les fenêtres
E3	Détection de mouvement	Détecter la présence ou le déplacement d'une personne dans la zone surveillée.	F3	Capteur PIR	Détection de mouvement infrarouge
E4	Activation avec code secret	L'utilisateur doit pouvoir activer le système via un code.	F4	Clavier / Authentification	Saisie du code utilisateur
E5	Désactivation avec code	Possibilité de désarmer l'alarme avec un code valide.	F5	Microcontrôleur	Traitement des données des capteurs
E6	Signal sonore	Émettre une alarme sonore lors d'une intrusion.	F6	Module IoT (MQTT / Node-RED)	Transmission des données
E7	Notification / affichage	Informar l'utilisateur de l'origine de l'intrusion.	F7	Sirène	Alarme sonore
E8	Fiabilité	Le système doit fonctionner sans erreur majeure.	F8	Interface utilisateur	Tableau de bord / affichage
E9	Coût réduit	La solution doit rester économique.	F9	Alimentation	Gestion de l'énergie
E10	Facilité d'entretien	Maintenance simple et rapide.	F10	Boîtier	Protection matérielle

Exigences \ Fonctions	F1	F2	F3	F4	F5	F6	F7		Valeur	Signification
E1 Détection porte	5				3	2			0	Pas de relation
E2 Détection fenêtre		5			3	2			1	Relation faible
E3 Détection mouvement			5		3	2			3	Relation moyenne
E4 Activation code				5	4				5	Relation forte
E5 Désactivation				5	4					
E6 Signal sonore					3		5			
E7 Notification					3	5				
E8 Fiabilité	3	3	3	3	5	4	4			
E9 Coût	3	3	3	3	3	3	3			
E10 Maintenance					3	3				

Figure 3 : Matrice DPF liant les besoins utilisateurs aux fonctions de service.

II. 4. Définition du périmètre : Le Diagramme de Contexte

Le diagramme de contexte permet de délimiter le périmètre de notre solution et d'identifier les entités externes (acteurs ou systèmes) qui interagissent avec elle. Cette étape est cruciale pour garantir que tous les flux d'informations ont été pris en compte lors de la conception.

Dans notre architecture, le système de surveillance centrale interagit avec quatre entités principales :

1. **Le Patient** : Il est la source primaire de la donnée. Ses constantes physiologiques (température, rythme cardiaque, etc.) sont captées et injectées dans le système.
2. **Le Personnel Médical (Médecins/Infirmiers)** : C'est l'utilisateur final. Il reçoit les alertes en temps réel et consulte les données via le Dashboard de supervision pour la prise de décision clinique.
3. **L'Administrateur Système** : Il assure la maintenance technique, la configuration des seuils d'alerte et la gestion de la connectivité réseau.
4. **Le Système de Stockage (Base de données CSV)** : Le système y exporte les données historisées pour permettre des analyses ultérieures et l'entraînement de modèles d'IA.

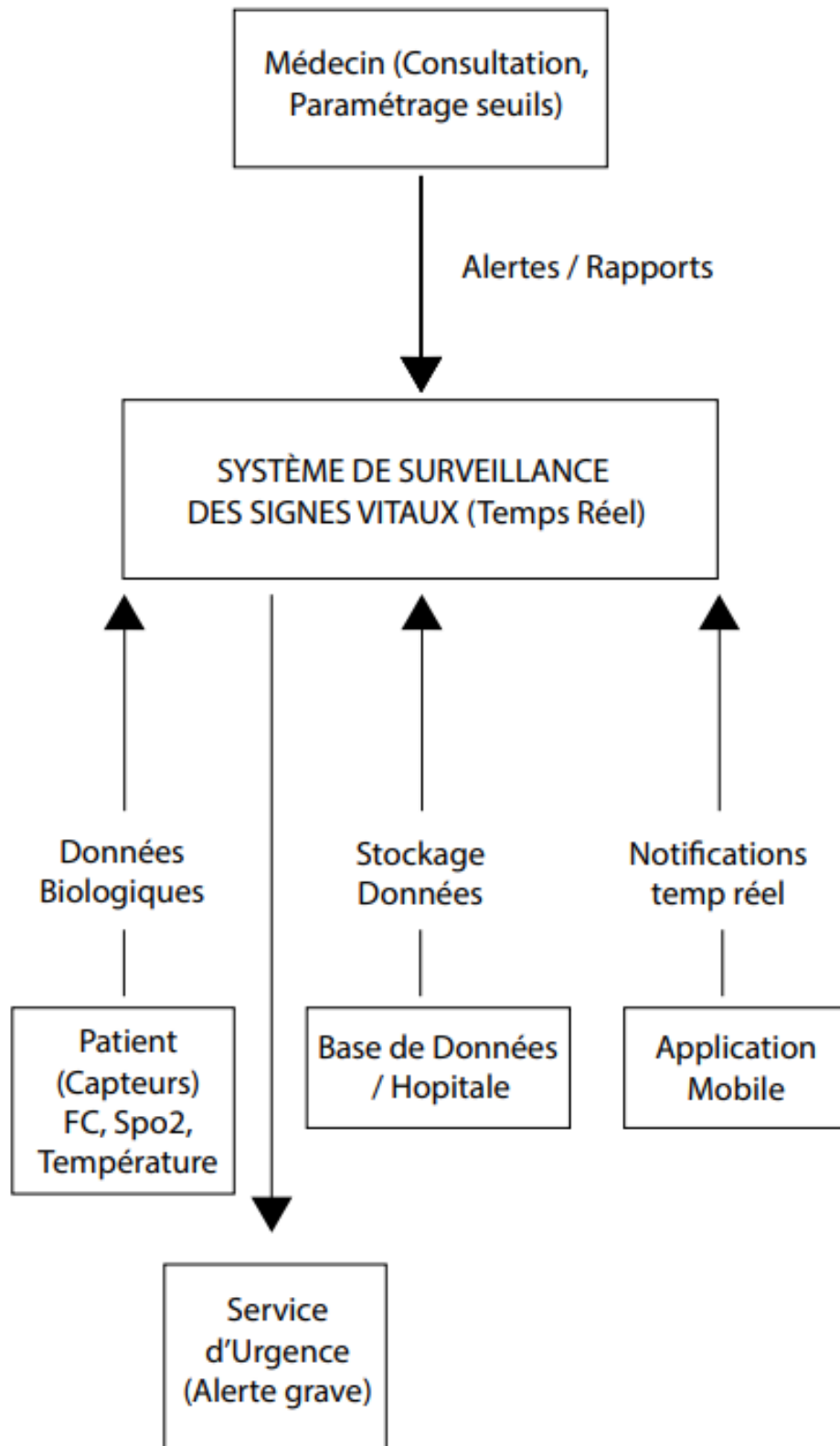


Figure 4 : Diagramme de contexte du système de surveillance intelligente.

II. 5. Architecture du système : Diagramme Physique

Le diagramme physique représente l'infrastructure matérielle et réseau de notre solution. Il illustre la connectivité entre l'unité d'acquisition (ESP32), le réseau Wi-Fi, le serveur de messagerie (HiveMQ) et les terminaux de supervision (Node-RED et Python). Ce schéma est essentiel pour comprendre la topologie "Edge-to-Cloud" de notre système de surveillance.

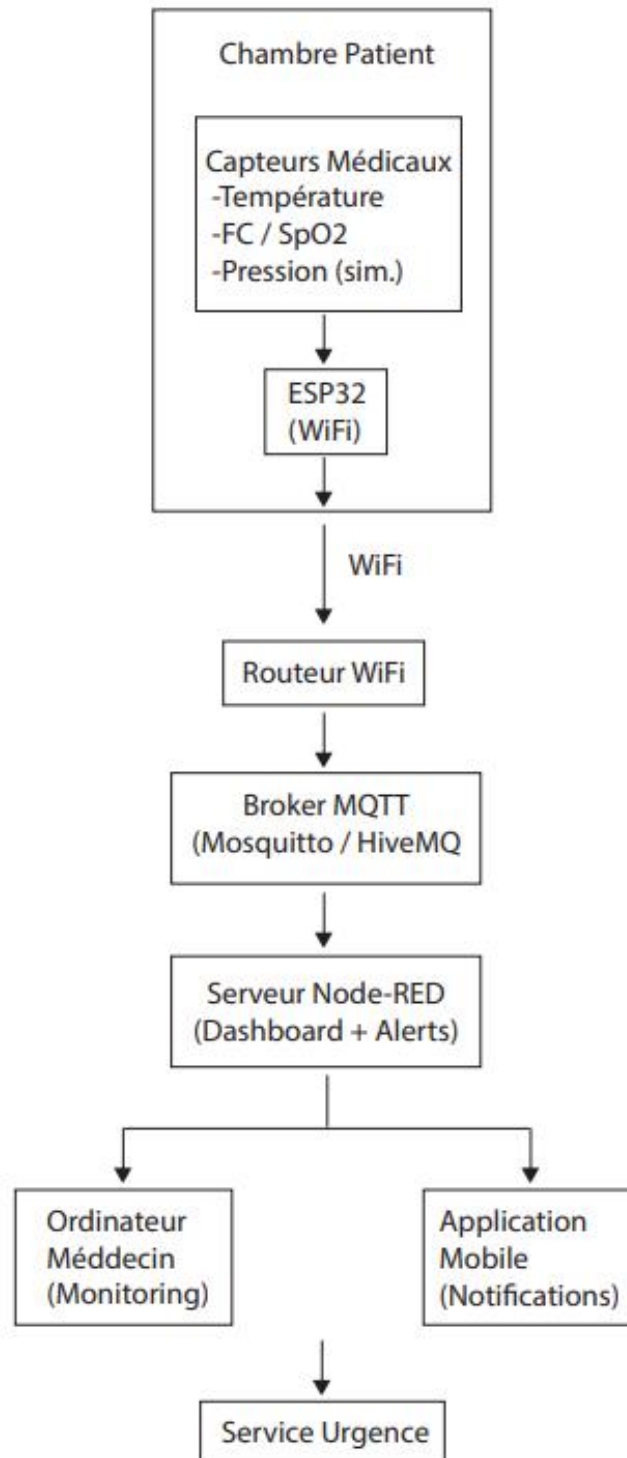


Figure 5 : Architecture physique et schéma de connectivité IoT.

CHAPITRE III :
ENVIRONNEMENT MATERIEL ET LOGICIEL

III. 1. Architecture Matérielle (Simulation Wokwi)

Le cœur du système repose sur une unité d'acquisition intelligente capable de traiter et de transmettre des données en temps réel.

<https://wokwi.com/>

III. 1. 1. Le Microcontrôleur : ESP32

L'ESP32 a été choisi pour sa puissance de calcul et, surtout, pour ses capacités de communication **Wi-Fi et Bluetooth** intégrées. Dans notre projet, il joue le rôle de passerelle (gateway) : il lit les signaux des capteurs, les convertit en format numérique et les encapsule dans des messages MQTT.

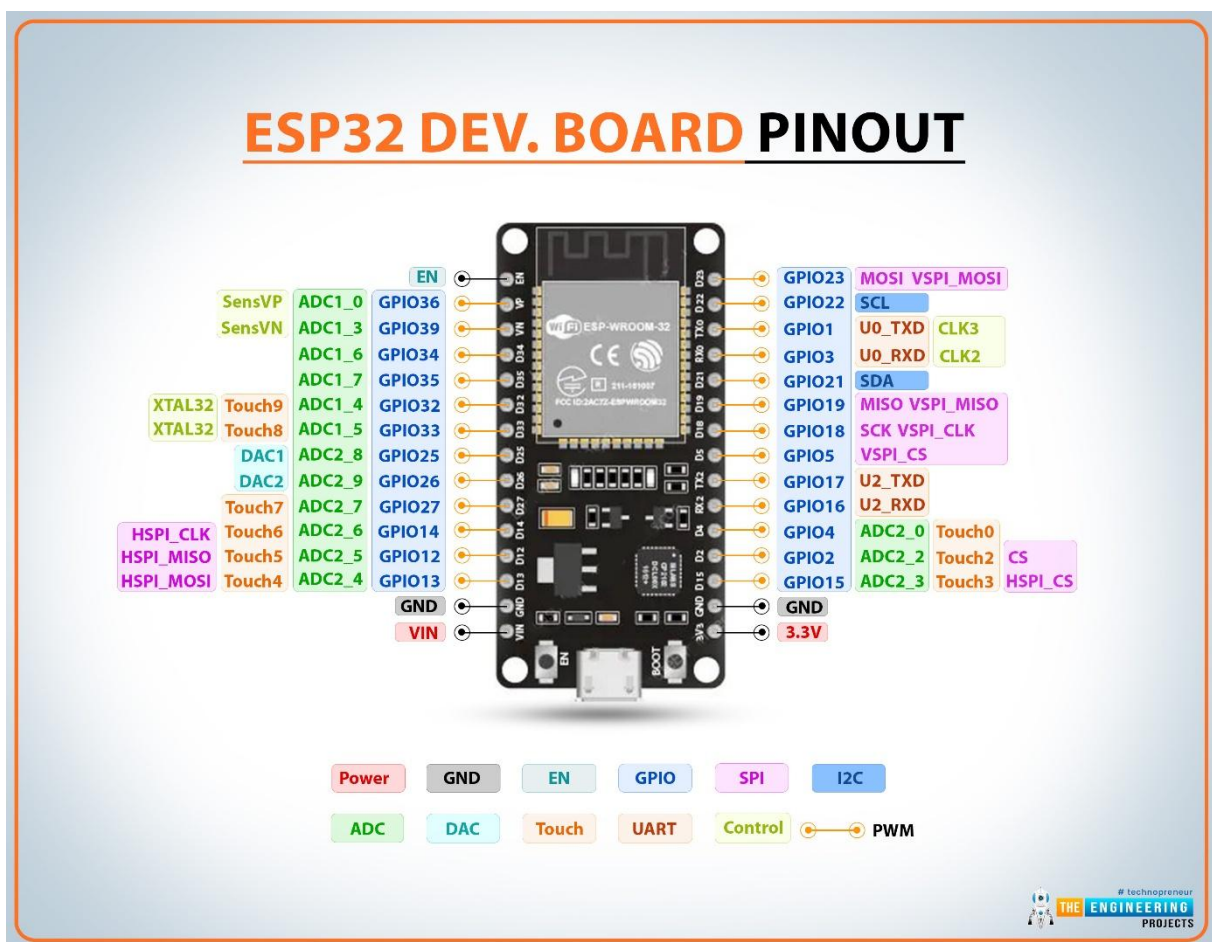


Figure 6 : ESP32

III. 1. 2. Les Capteurs de Constantes Vitales

Pour simuler un environnement hospitalier réaliste, nous avons intégré plusieurs capteurs :

- **Capteur de Température Corporelle (DS18B20)** : Utilisé pour sa haute précision. Il fonctionne avec le protocole *OneWire*, permettant une mesure stable de la température du patient.



Figure 7 : DS18B20 .

- **Capteur d'Ambiance (DHT22)** : Mesure la température et l'humidité de la chambre d'hôpital, des paramètres cruciaux pour le confort et la santé respiratoire.

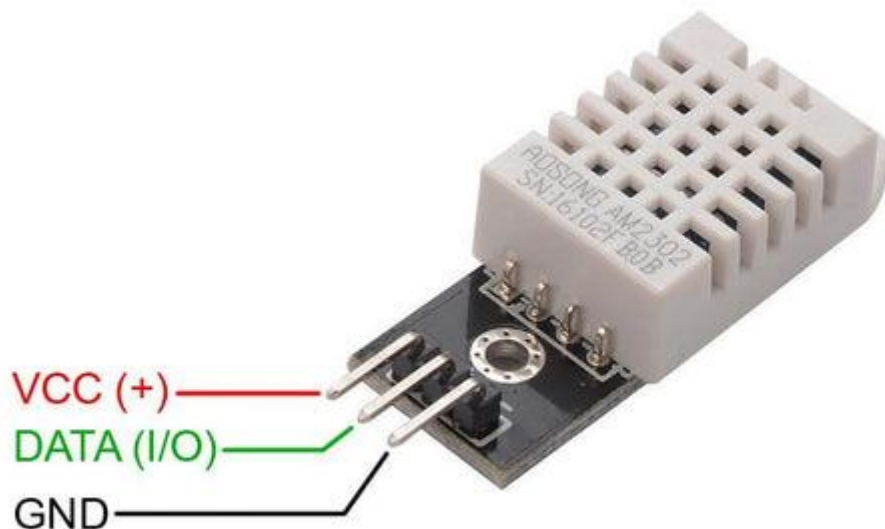


Figure 8 : DHT22 .

- **Simulateurs Physiologiques (Potentiomètres)** : Afin de simuler dynamiquement des paramètres variables comme la **Fréquence Cardiaque (BPM)** et le **Taux d'Oxygène (SpO2)**, nous utilisons des

entrées analogiques. Cela nous permet de tester les réactions du système face à des situations d'urgence (tachycardie ou hypoxie).

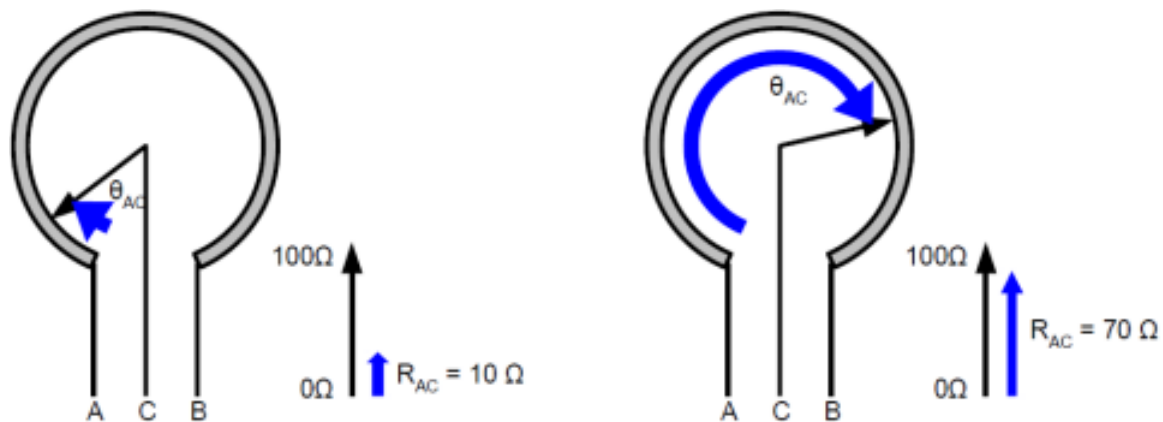


Figure 9 : Simulateurs Physiologiques (Potentiomètres) .

III. 2. Environnement Logiciel

La fluidité du système dépend d'une chaîne logicielle intégrée, allant de la simulation à l'analyse de données.

III. 2. 1. Wokwi : Simulateur IoT

Wokwi est la plateforme de simulation utilisée pour concevoir le circuit électronique sans contraintes matérielles physiques. Elle permet de tester le code C++ (Arduino) de l'ESP32 et de vérifier la connectivité Wi-Fi virtuelle.

III. 2. 2. Node-RED : Plateforme de Supervision

Node-RED est un outil de programmation visuelle basé sur les flux (flow-based programming). Il est utilisé pour :

- Réceptionner et décoder les messages MQTT.
- Concevoir l'interface utilisateur (**Dashboard**).
- Gérer la logique des alertes médicales (visuelles et sonores).

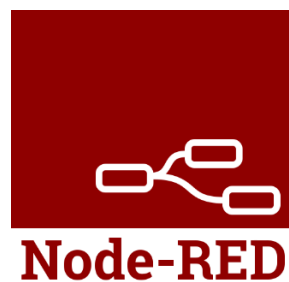


Figure 10 : Node-RED .

III. 2. 3. Broker MQTT : HiveMQ

Le protocole **MQTT** (Message Queuing Telemetry Transport) est le standard de l'IoT. Nous utilisons le broker public de **HiveMQ** pour assurer le transit des données entre l'ESP32 et nos interfaces de supervision. Sa légèreté garantit une transmission quasi instantanée des alertes critiques.

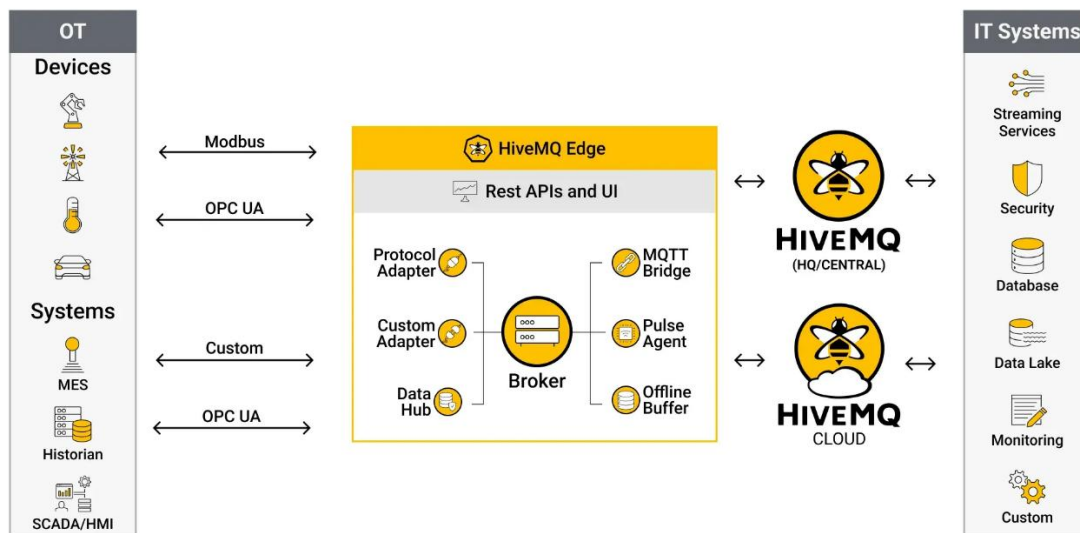


Figure 11 : Le broker public de HiveMQ .

III. 2. 4. Python : Analyse et Intelligence Artificielle

Pour la partie analytique, nous utilisons **Python** (via VS Code). Grâce à ses bibliothèques puissantes, Python agit comme un cerveau secondaire qui écoute les données, détecte les tendances anormales et permet l'historisation des données dans une base de données locale (**Fichier CSV**).

CHAPITRE IV :

ARCHITECTURE ET PROTOCOLE MQTT

IV. 1. Architecture Globale du Système

Notre solution repose sur une architecture en trois couches distinctes, garantissant une séparation claire entre l'acquisition, le transport et le traitement des données :

IV. 1. 1. Couche de Perception (Edge) :

Constituée de l'ESP32 et des capteurs. Elle assure la capture des constantes vitales et leur conversion en messages numériques.

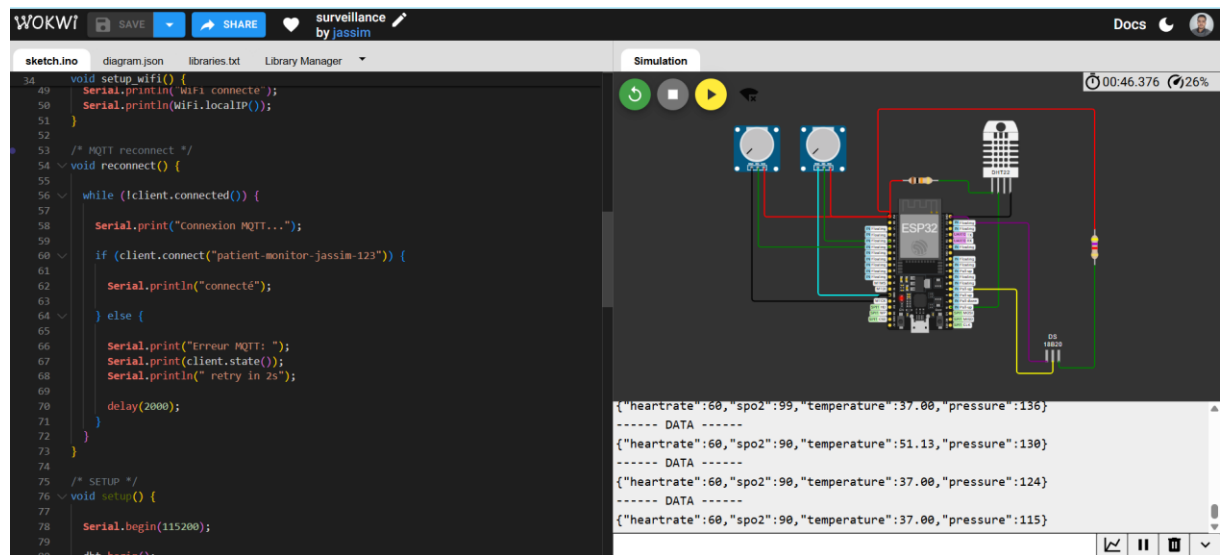


Figure 12 : Architecture du système ESP32 et les capteurs et le code utiliser sous dessous .

IV. 1. 2. Le code :

```
1  #include <WiFi.h>
2  #include <PubSubClient.h>
3  #include <DHT.h>
4  #include <OneWire.h>
5  #include <DallasTemperature.h>
6
7  /* WIFI */
8  const char* ssid = "wokwi-GUEST";
9  const char* password = "";
10
11 /* MQTT Broker */
12 const char* mqtt_server = "broker.hivemq.com";
13
14 WiFiClient espClient;
15 PubSubClient client(espClient);
16
17 /* DHT22 */
18 #define DHTPIN 15
19 #define DHTTYPE DHT22
20 DHT dht(DHTPIN, DHTTYPE);
21
22 /* DS18B20 */
23 #define ONE_WIRE_BUS 4
24 OneWire oneWire(ONE_WIRE_BUS);
25 DallasTemperature sensors(&oneWire);
26
27 /* Potentiometers */
28 #define HR_PIN 34
29 #define SPO2_PIN 35
30
31 long lastMsg = 0;
```

```

32
33  /* WIFI CONNECTION */
34  void setup_wifi() {
35
36      delay(10);
37      Serial.println();
38      Serial.print("Connexion WiFi: ");
39      Serial.println(ssid);
40
41      WiFi.begin(ssid, password);
42
43      while (WiFi.status() != WL_CONNECTED) {
44          delay(500);
45          Serial.print(".");
46      }
47
48      Serial.println("");
49      Serial.println("WiFi connecté");
50      Serial.println(WiFi.localIP());
51  }
52
53  /* MQTT reconnect */
54  void reconnect() {
55
56      while (!client.connected()) {
57
58          Serial.print("Connexion MQTT...");
59
60          if (client.connect("patient-monitor-jassim-123")) {
61
62              Serial.println("connecté");

```

```

63          Serial.println("connecté");
64      } else {
65
66          Serial.print("Erreur MQTT: ");
67          Serial.print(client.state());
68          Serial.println(" retry in 2s");
69
70          delay(2000);
71      }
72  }
73  }
74
75  /* SETUP */
76  void setup() {
77
78      Serial.begin(115200);
79
80      dht.begin();
81      sensors.begin();
82
83      setup_wifi();
84
85      client.setServer(mqtt_server, 1883);
86
87      randomSeed(analogRead(0));
88  }
89
90  /* LOOP */
91  void loop() {
92

```

```

93     if (!client.connected()) reconnect();
94     client.loop();
95
96     long now = millis();
97
98     if (now - lastMsg > 3000) {
99
100         lastMsg = now;
101
102         /* DHT22 Temperature */
103         float tempAir = dht.readTemperature();
104
105         /* DS18B20 Temperature */
106         sensors.requestTemperatures();
107         float tempBody = sensors.getTempCByIndex(0);
108
109         /* Heart Rate simulation */
110         int heartRate = map(analogRead(HR_PIN), 0, 4095, 60, 140);
111
112         /* SpO2 simulation */
113         int spo2 = map(analogRead(SPO2_PIN), 0, 4095, 90, 100);
114
115         /* Blood pressure simulated */
116         int pressure = random(110, 140);
117
118         /* JSON Payload */
119         String payload = "{";
120         payload += "\"heartrate\":" + String(heartRate) + ",";
121         payload += "\"spo2\":" + String(spo2) + ",";
122         payload += "\"temperature\":" + String(tempBody) + ",";
123         payload += "\"pressure\":" + String(pressure);

```

```

124         payload += "}";
125
126         /* MQTT Publish */
127         client.publish("hospital/patient1/vitals", payload.c_str());
128
129         /* Serial Monitor */
130         Serial.println("----- DATA -----");
131         Serial.println(payload);
132     }
133 }

```

IV. 1. 3. Couche de Transport (Cloud) :

Représentée par le **Broker MQTT (HiveMQ)**. Elle agit comme un centre de tri intelligent qui reçoit les messages et les redistribue instantanément.



Need a fully managed MQTT broker?
Get your own Cloud broker and connect up to 100 devices for free.

[Get your free account](#)

Connection

● connected

Host
broker.hivemq.com

Port
8884

ClientID
patient1-jassim-99

Disconnect

Username

Password

Keep Alive
60

SSL
x

Clean Session
x

Last-Will Topic

Last-Will QoS
0

Last-Will Retain
☐

Last-Will Message

Publish

Topic
testtopic/1

QoS
0

Retain
☐

Publish

Message

Subscriptions

Add New Topic Subscription

Qos: 0

hospital/patient1/vitals

Messages

2026-04-04 16:16:07 Topic: hospital/patient1/vitals Qos: 0

{ "heartrate": 60, "spo2": 90, "temperature": 37.00, "pressure": 115 }

Figure 13 : Couche de Transport (Cloud) .

IV. 1. 4. Couche d'Application (User Interface & AI) :

Comprend le **Dashboard Node-RED** pour la surveillance humaine et le script **Python** pour l'analyse prédictive.

```

Al.py > ...
1  import paho.mqtt.client as mqtt
2  from paho.mqtt.client import CallbackAPIVersion
3  import json
4  import time
5
6  # --- CONFIGURATION ---
7  BROKER = "broker.hivemq.com"
8  PORT = 1883
9  TOPIC = "hospital/patient1/vitals"
10
11 # --- MÉMOIRE DE L'IA ---
12 # Liste pour stocker l'historique récent (les 5 dernières mesures)
13 historique_fc = []
14
15 def analyse_tendance(historique):
16     """
17     Algorithme de prédiction basique (Analyse de tendance).
18     Compare la moyenne la plus ancienne à la plus récente.
19     """
20     if len(historique) < 5:
21         return "Collecte des données pour l'analyse..."
22
23     # Séparation des données pour voir l'évolution
24     moyenne_ancienne = sum(historique[:2]) / 2 # Les 2 plus anciennes
25     moyenne_recente = sum(historique[-2:]) / 2 # Les 2 plus récentes

```

Figure 14 : Code utiliser pour le modèle de AI .

```

27     evolution = moyenne_recente - moyenne_ancienne
28
29     # Logique d'alerte prédictive
30     if evolution > 15:
31         return "PRÉDICTION : Hausse soudaine détectée. Risque de tachycardie imminent !"
32     elif evolution < -15:
33         return "↘ Baisse soudaine du rythme cardiaque."
34     elif evolution > 5:
35         return "↗ Tendance légèrement à la hausse."
36     else:
37         return "Rythme stable."
38
39 def on_connect(client, userdata, flags, reason_code, properties):
40     if reason_code == 0:
41         print("Moteur IA connecté au réseau de l'hôpital.")
42         client.subscribe(TOPIC)
43
44 def on_message(client, userdata, msg):
45     try:
46         data = json.loads(msg.payload.decode())
47         fc_actuelle = data.get("heartrate")
48
49         if fc_actuelle is not None:
50             # 1. Ajouter la nouvelle valeur à la mémoire

```

```

51     historique_fc.append(fc_actuelle)
52
53     # 2. Ne garder que les 5 dernières valeurs (fenêtre glissante)
54     if len(historique_fc) > 5:
55         historique_fc.pop(0)
56
57     # 3. Lancer l'analyse prédictive
58     diagnostic_ia = analyse_tendance(historique_fc)
59
60     print("-" * 50)
61     print(f"Mesure actuelle : {fc_actuelle} bpm")
62     print(f"Analyse IA      : {diagnostic_ia}")
63
64 except json.JSONDecodeError:
65     pass
66

```

```

67 # Initialisation du client MQTT
68 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, "python-ia-predictive-99")
69 client.on_connect = on_connect
70 client.on_message = on_message
71
72 client.connect(BROKER, PORT, 60)
73
74 print("Lancement de l'analyse prédictive des signes vitaux...")
75 client.loop_forever()

```

IV. 1. 5. Résultat :

```

-----
Mesure actuelle : 60 bpm
Analyse IA      : PRÉDICTION : Hausse soudaine détectée. Risque de tachycardie imminent !
-----
Mesure actuelle : 60 bpm
Analyse IA      : \ Baisse soudaine du rythme cardiaque.
-----
Mesure actuelle : 60 bpm
Analyse IA      : \ Baisse soudaine du rythme cardiaque.
-----

```

```

Analyse IA      : Rythme stable.
-----
Mesure actuelle : 140 bpm
Analyse IA      : PRÉDICTION : Hausse soudaine détectée. Risque de tachycardie imminent !
-----
Mesure actuelle : 140 bpm
Analyse IA      : PRÉDICTION : Hausse soudaine détectée. Risque de tachycardie imminent !
-----
Mesure actuelle : 140 bpm
Analyse IA      : PRÉDICTION : Hausse soudaine détectée. Risque de tachycardie imminent !
-----

```

IV. 2. Le Protocole MQTT :

Le standard de l'IoT Médical

Le choix du protocole **MQTT (Message Queuing Telemetry Transport)** est stratégique. Contrairement au HTTP classique, le MQTT est extrêmement léger et fonctionne sur un modèle **Subscribe** (Publication/Abonnement).

- **Légereté** : Les entêtes des messages sont minimalistes, ce qui permet une transmission ultra-rapide même avec une connexion Wi-Fi instable.

- **Temps Réel** : La latence est quasi nulle, ce qui est indispensable pour détecter une anomalie cardiaque en quelques millisecondes.
- **Efficacité** : Le système ne consomme de la bande passante que lorsqu'une donnée est envoyée, optimisant ainsi les ressources réseau de l'hôpital.

IV. 2. 1. Le code :

```

1  import paho.mqtt.client as mqtt
2  from paho.mqtt.client import CallbackAPIVersion
3  import json
4
5  BROKER = "broker.hivemq.com"
6  PORT = 1883
7  PATIENT_ID = "patient1"
8  # Le nouveau topic unique envoyé par votre ESP32
9  TOPIC = f"hospital/{PATIENT_ID}/vitals"
10
11 # Seuils critiques
12 SEUIL_FC = 120
13 SEUIL_TEMP = 39.0
14
15 # Callback de connexion
16 def on_connect(client, userdata, flags, reason_code, properties):
17     if reason_code == 0:
18         print(" Connecté au Broker MQTT avec succès !")
19         # S'abonner au topic JSON unique
20         client.subscribe(TOPIC)
21         print(f" En attente des données sur le topic : `{TOPIC}`")
22     else:
23         print(f" Échec de la connexion, code de retour {reason_code}")
24

```

Figure 15 : Code utiliser pour le subscriber .

```

25 # Callback de réception des messages
26 def on_message(client, userdata, msg):
27     try:
28         # Décodage du message brut (string) en dictionnaire Python (JSON)
29         payload_str = msg.payload.decode()
30         data = json.loads(payload_str)
31
32         # Extraction des valeurs
33         fc = data.get("heartrate")
34         spo2 = data.get("spo2")
35         temp = data.get("temperature")
36         pression = data.get("pressure")
37
38         # Affichage structuré
39         print("-" * 40)
40         print(f"📡 Données reçues pour {PATIENT_ID} :")
41         print(f"❤️ FC : {fc} bpm")
42         print(f"💧 SpO2 : {spo2} %")
43         print(f"🌡️ Temp : {temp} °C")
44         print(f"💓 Pression : {pression} mmHg")
45
46         # --- LOGIQUE D'ALERTE AUTOMATIQUE ---
47         if fc is not None and fc > SEUIL_FC:
48             print("ALERTE CRITIQUE : Fréquence cardiaque trop élevée ! ")
49         if temp is not None and temp > SEUIL_TEMP:

```

```

50         print("    ALERTE CRITIQUE : Température corporelle trop élevée ! ")
51
52     except json.JSONDecodeError:
53         print(f" Erreur de lecture JSON. Message brut reçu : {msg.payload.decode()}")
54     except Exception as e:
55         print(f" Erreur inattendue : {e}")
56
57 # IMPORTANT : Utilisation d'un ID client unique pour éviter les conflits et déconnexions
58 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, "python-analyzer-jassim-9988")
59
60 client.on_connect = on_connect
61 client.on_message = on_message
62
63 # Connexion au broker public
64 client.connect(BROKER, PORT, 60)
65
66 # Boucle infinie pour écouter les messages
67 try:
68     client.loop_forever()
69 except KeyboardInterrupt:
70     print("\n🛑 Arrêt du script par l'utilisateur.")
71     client.disconnect()

```

IV. 2. 2. Résultat :

```

💧 SpO2      : 100 %
🌡️ Temp     : 37.0 °C
❤️ Pression : 115 mmHg
ALERTE CRITIQUE : Fréquence cardiaque trop élevée !
-----
📡 Données reçues pour patient1 :
❤️ FC      : 60 bpm
💧 SpO2     : 90 %
🌡️ Temp    : 37.0 °C
❤️ Pression : 123 mmHg
-----

```

IV. 2. 3. Organisation des Flux (Topics)

Pour organiser les données, nous utilisons une structure de "Topics" (sujets) hiérarchisée. Cela permet de cibler précisément quel patient est surveillé.

- **Topic utilisé** : hospital/patient1/vitals
- **Le Publisher (L'émetteur)** : L'ESP32 publie les données sur ce topic toutes les 3 secondes.
- **Les Subscribers (Les récepteurs)** : Node-RED et le script Python sont abonnés à ce même topic pour traiter l'information simultanément.

IV. 2. 4. Format de Données : Le JSON

Pour que les différentes machines (ESP32, Serveur, Python) se comprennent parfaitement, nous utilisons le format **JSON (JavaScript Object Notation)**. C'est un format de texte léger et universel.

Exemple d'un message envoyé par notre système :

JSON

```
{
```

```
"heartrate": 72,  
"spo2": 98,  
"temperature": 36.5,  
"pressure": 120  
}
```

Ce format permet d'encapsuler toutes les constantes vitales dans un seul "paquet" de données, facilitant ainsi le décodage et l'affichage immédiat sur le tableau de bord.

IV. 2. 5. Fiabilité et Sécurité (QoS)

Pour garantir que les alertes critiques ne soient jamais perdues, le protocole permet de définir une **Qualité de Service (QoS)**. Dans notre architecture, nous privilégions la réception continue pour assurer que le flux de données vers le Dashboard soit constant, permettant aux infirmiers d'avoir une vision fluide de l'état du patient.

CHAPITRE V :

REALISATION DU DASHBOARD (NODE-RED)

V. 1. Philosophie de conception sous Node-RED

Node-RED permet de transformer des flux de données complexes en une interface visuelle simplifiée. Notre architecture logicielle au sein de Node-RED repose sur trois piliers :

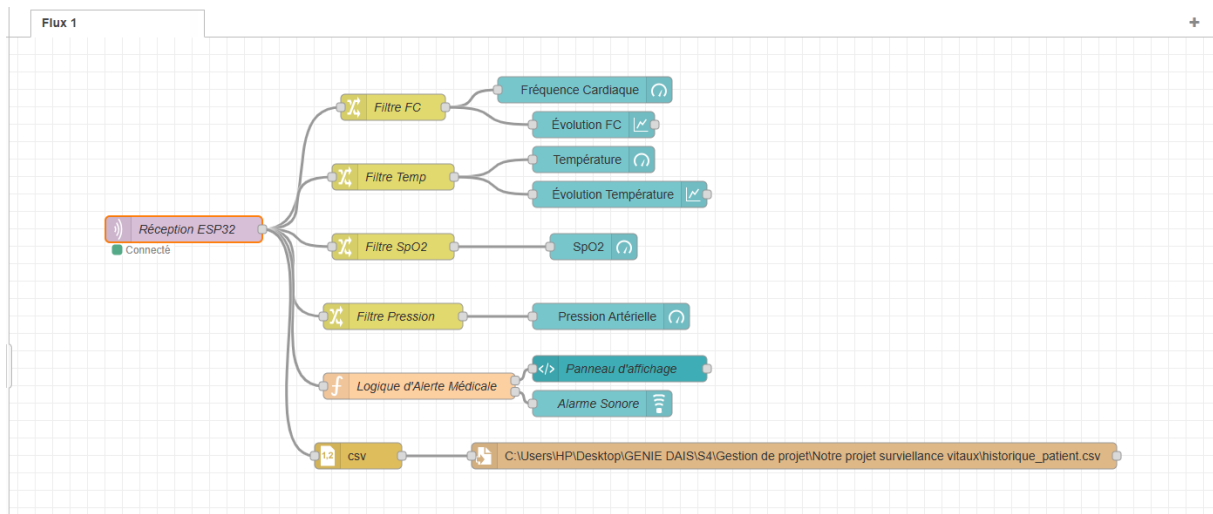


Figure 16 : Architecture logicielle au sein de Node-RED .

- **La Réception** : Interception des paquets MQTT provenant du broker HiveMQ.

V. 1. 1. Nœud de Réception (Entrée)

Nom : Réception ESP32 (MQTT in)

- **Serveur** : broker.hivemq.com
- **Port** : 1883
- **Topic** : hospital/patient1/vitals
- **Output** : parsed JSON object (C'est ce qui permet aux filtres suivants de lire les variables directement).
- **Le Traitement** : Décodage du format JSON et filtrage des variables (Fréquence cardiaque, SpO2, etc.).

V. 1. 2. Nœuds de Filtrage (Change Nodes)

Ces nœuds servent à isoler chaque donnée du paquet JSON pour l'envoyer vers les graphiques correspondants.

Filtre FC :

- **Définir** : msg.payload
- **Sur la valeur** : msg.payload.hearttrate

Filtre Temp :

- **Définir** : msg.payload

- **Sur la valeur :** msg.payload.temperature

Filtre SpO2 :

- **Définir :** msg.payload
- **Sur la valeur :** msg.payload.spo2

Filtre Pression :

- **Définir :** msg.payload
- **Sur la valeur :** msg.payload.pressure

V. 1. 3. Nœud de Traitement Intelligent (Function)

Nom : Logique d'Alerte Médicale

Code (JavaScript) :

```
var hr = msg.payload.heartrate;
var sp = msg.payload.spo2;
var temp = msg.payload.temperature;
var pr = msg.payload.pressure
var statusMsg = "";
var color = "#00b500"; // Vert par défaut

if (hr > 120 || temp > 39 || sp > 95 || pr > 120 ) {
    statusMsg = "⚠ ALERTE CRITIQUE : INTERVENTION REQUISE";
    color = "#ca3838"; // Rouge
    // Déclencher le son
    node.send([null, {payload: "Alerte médicale, veuillez vérifier le patient Jassim ID442434."}]);
} else {
    statusMsg = "☑ Constantes Normales";
}

var html = "<div style='background-color:" + color + "; color:white; padding:20px; text-align:center; font-size:24px; font-weight:bold; border-radius:10px;'>" + statusMsg + "</div>";

return [{payload: html}, null];
```

V. 1. 4. Nœud de Formatage de Données (CSV)

Nom : csv

Columns : `heartrate,spo2,temperature,pressure`

Separator : `semicolon` (;) [Pour une compatibilité parfaite avec Excel].

Options : "Include column name row" (Coché).

- **Output :** `a single multicolumn string`.

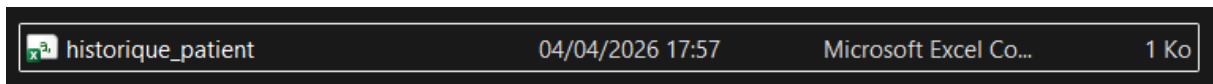


Figure 17 : Le fichier CSV sauvegarder sur notre path .

V. 1. 5. Nœud de Stockage (File out)

- **Nom :** Historique Patient (Path complet)
- **Filename :** `C:\Users\HP\Desktop\GENIE DAIS\S4\Gestion de projet\Notre projet surveillance vitaux\historique\_patient.csv`
- **Action :** `append to file` (Ajouter à la suite du fichier).
- **Add newline (\n) :** Coché (Pour que chaque mesure soit sur une nouvelle ligne).

V. 1. 6. Nœuds d'Interface (Dashboard)

En utilisent les configurations suivantes :

- **Gauges :** Paramétrées avec des secteurs de couleurs (Vert : 60-100, Jaune : 100-120, Rouge : >120).
- **Charts :** Configurés pour afficher les 10 dernières minutes de données ("Last 10 minutes").
- **Alarme Sonore :** Nœud `audio out` configuré pour jouer un "Bip" lors de la réception d'un message d'alerte.
- **La Restitution :** Affichage dynamique sur un tableau de bord (Dashboard) accessible via un navigateur web.



Figure 18 : Interface (Dashboard) .

V. 2. Architecture du flux de données (Flow)

Le flux a été structuré de manière modulaire pour garantir une haute disponibilité du système. Chaque donnée vitale suit un cheminement précis :

V. 2. 1. Nœud MQTT In :

Souscrit au topic hospital/patient1/vitals et récupère le message brut.

V. 2. 2. Nœud JSON :

Convertit la chaîne de caractères reçue en un objet JavaScript exploitable par les nœuds suivants.

V. 2. 3. Nœuds de Fonction (Switch/Change) :

Séparent les différentes mesures pour les diriger vers leurs afficheurs respectifs.

V. 2. 4. Composants du Dashboard Médical

L'interface a été conçue pour être intuitive et permettre une lecture rapide des constantes, même en situation d'urgence :

- **Les Gauges (Jauges) :** Elles offrent une vue instantanée du niveau des constantes. Nous avons configuré des codes couleurs sémantiques :
 - **Vert :** Zone normale (ex: 60-100 bpm).
 - **Jaune :** Zone de vigilance.
 - **Rouge :** Zone critique nécessitant une intervention immédiate.

- **Les Charts (Graphiques temporels) :** Ils permettent de visualiser l'évolution des paramètres (Température, Fréquence cardiaque) sur une période donnée, aidant ainsi à détecter des anomalies cycliques ou des dégradations progressives.
- **Panneau d'affichage textuel :** Résume l'état général du patient et affiche les notifications du système.

V. 3. Logique d'Alerte et de Sécurité

La sécurité du patient est assurée par un nœud de **Logique d'Alerte Médicale**. Ce module surveille en permanence si les données dépassent les seuils de sécurité pré-établis (ex: Fréquence cardiaque > 120 bpm ou Température > 38.5°C).

En cas de dépassement, le système déclenche automatiquement :

- **Une alerte visuelle :** Changement de couleur des widgets sur le dashboard.
- **Une notification sonore :** Activation d'un signal d'alarme pour attirer l'attention de l'infirmier, même s'il ne regarde pas l'écran.
- **Un message d'état :** Affichage précis de la nature du problème (ex : Alerte médicale, veuillez vérifier le patient Jassim ID442434.").

Grâce à cette interface, la donnée brute issue du microcontrôleur devient une information clinique exploitable. Le Dashboard transforme l'IoT en un véritable outil d'aide à la décision médicale.

CHAPITRE VI :

IA ET HISTORISATION DES DONNEES

VI. 1. Stratégie d'Historisation (Data Logging)

Pour qu'un système médical soit homologué, la traçabilité est une obligation légale. Nous avons mis en place un module d'archivage automatique via Node-RED.

- **Le format CSV (Comma-Separated Values) :** Nous avons choisi ce format pour sa légèreté et sa compatibilité universelle avec les logiciels d'analyse (Excel, Pandas).
- **Structure de la base de données :** Chaque ligne du fichier représente une capture instantanée de l'état du patient, structurée en quatre colonnes : heartrate, spo2, temperature, et pressure.
- **Utilité pour le Machine Learning :** Au-delà du simple archivage, ce fichier constitue un **Dataset** (jeu de données). En accumulant ces mesures sur plusieurs jours, il devient possible d'entraîner des modèles d'IA plus complexes (réseaux de neurones ou forêts aléatoires) pour reconnaître des signatures pathologiques spécifiques à chaque patient.

VI. 2. Module d'Analyse Prédictive en Python

L'originalité de notre approche réside dans l'utilisation d'un script Python indépendant, agissant comme un "expert virtuel" qui écoute le flux MQTT en permanence.

VI. 2. 1. L'Algorithme de Fenêtre Glissante (Sliding Window)

Contrairement aux alertes classiques qui ne regardent qu'un chiffre à l'instant t , notre script Python utilise une **mémoire à court terme**. Il stocke les 5 dernières mesures reçues dans une liste dynamique.

L'algorithme calcule ensuite la variation (la pente) entre la moyenne des données anciennes et celle des données les plus récentes.

VI. 2. 2. Détection de Tendance (Trend Analysis)

Cette approche permet de détecter une **accélération** ou une **chute brutale** avant même que les seuils critiques ne soient atteints :

- **Analyse de stabilité :** Si la variation est proche de zéro, l'IA affiche " Rythme stable".
- **Alerte prédictive :** Si une hausse soudaine est détectée (ex: passage rapide de 70 à 110 bpm), l'IA génère un avertissement : "*PRÉDICTION : Hausse soudaine détectée*". Cela permet au personnel soignant d'intervenir quelques minutes plus tôt, ce qui est souvent vital.

VI. 3. Vers une Médecine Proactive

Grâce à ce chapitre, nous démontrons le passage de la **médecine réactive** (soigner quand la crise arrive) à la **médecine proactive** (anticiper la crise). L'intégration de ce script Python prouve que l'IoT, lorsqu'il est couplé à l'analyse de données, devient un outil de diagnostic puissant.

L'historisation assure la mémoire du système, tandis que l'analyse prédictive assure sa capacité d'anticipation. Cet ensemble technologique répond parfaitement aux exigences de l'Ingénierie Digitale appliquée à la Santé 4.0.

VI. 4. Conclusion Générale et Perspectives

VI. 4. 1. Bilan du Projet

La boucle est bouclée : ce projet nous a permis de concevoir et de réaliser un système complet de surveillance hospitalière intelligente, allant de la capture du signal physiologique jusqu'à l'analyse prédictive.

En intégrant un microcontrôleur **ESP32**, le protocole **MQTT**, une interface **Node-RED** et un moteur d'analyse en **Python**, nous avons démontré qu'il est possible de transformer des données brutes en informations cliniques cruciales. Le succès de ce projet réside dans sa capacité à passer d'une **médecine réactive** (alerte sur seuil) à une **médecine proactive** (détection de tendances), offrant ainsi au personnel soignant un outil d'aide à la décision précieux et en temps réel.

VI. 4. 2. Apports Personnels

Sur le plan technique, ce travail a été l'occasion d'approfondir la maîtrise de l'écosystème IoT et de comprendre les défis liés à la latence et à la fiabilité des transmissions de données médicales. La manipulation de flux de données complexes et leur historisation sous format CSV m'ont permis de concrétiser mes acquis en ingénierie digitale et en intelligence artificielle, renforçant ma capacité à architecturer des solutions de santé connectée.

VI. 4. 3. Perspectives d'Évolution

Bien que le prototype actuel soit fonctionnel et robuste, plusieurs axes de développement peuvent être envisagés pour le porter vers un stade industriel :

1. **Sécurisation des Données (Cyber-santé) :** L'une des priorités futures serait l'implémentation d'un protocole **MQTTS (TLS/SSL)** pour garantir la confidentialité et l'intégrité des données médicales sensibles des patients.
2. **Intelligence Artificielle Avancée :** Grâce à l'historisation des données (fichiers CSV), nous pourrions entraîner des modèles de **Deep Learning** (type réseaux de neurones LSTM) capables de prédire des pathologies spécifiques (arythmies, apnées du sommeil) avec une précision bien supérieure à l'analyse de tendance actuelle.
3. **Scalabilité Multi-patients :** Faire évoluer l'architecture pour supporter la surveillance simultanée de plusieurs chambres, en automatisant la création dynamique de topics MQTT et de vues personnalisées sur le Dashboard.
4. **Déploiement Matériel Réel :** Passer de la simulation Wokwi à un prototype physique utilisant des capteurs biométriques réels (MAX30102 pour le SpO2/FC) afin de confronter le système aux bruits et aux contraintes du monde réel.

Mots de la fin : Ce projet n'est pas seulement une démonstration technologique ; c'est un premier pas vers l'hôpital de demain, où la technologie s'efface au profit de la précision des soins et de la sécurité des patients.